

Aula 3 — Interfaceamento paralelo e display

O controlador HD44780 e o custo da espera

Microcontroladores — DENE/UFMT

OBJETIVOS

- Reconhecer um barramento paralelo como um contrato de temporização entre linhas de dado, de endereço e de habilitação — e distingui-lo do acionamento direto de pino do encontro 2.
- Descrever a organização do controlador HD44780: memória de exibição, gerador de caracteres e a separação entre registrador de instrução e registrador de dados.
- Implementar a comunicação em quatro bits justificando cada atraso da sequência de inicialização a partir da folha de dados, e não por cópia.
- Estimar em ciclos de máquina o tempo consumido por uma atualização de tela e reconhecer o display como o periférico mais lento do sistema.
- Aplicar estratégias de atualização que evitam tremulação e reduzem o bloqueio do processador.

1 Do pino ao barramento

Acionar um pino foi, até aqui, a coisa mais simples do curso. Escrever 1 em LATD0 acende o LED, e ele fica aceso. O **nível** é a informação: quem observar o pino em qualquer instante lê o mesmo estado, e não existe momento privilegiado.

O display quebra isso, e essa é a mudança conceitual do encontro.

Escrever um valor em LATD não transfere nada para o display. O valor fica parado nos fios, e o controlador o ignora — até que uma segunda linha, a de habilitação, produza uma borda. É a borda que transfere. O nível apenas prepara.

Um barramento paralelo é um **contrato de temporização**. Ele reparte os fios em três papéis:

CONCEITO

dado — as linhas que carregam o valor (aqui, D7–D4);

endereço — a linha que diz o que o valor significa (RS: um bit de endereço, que escolhe entre dois registradores do controlador);

habilitação — a linha que declara o instante em que o dado é válido (E).

O contrato diz quanto tempo antes da borda o dado precisa estar estável, quanto tempo depois ele precisa permanecer, e qual é a largura mínima do pulso. Violar qualquer um desses números não produz erro: produz um caractere errado, às vezes.

O par de registradores selecionado por RS é a mesma ideia dos SFRs do encontro 1 — um registrador alcançado por um endereço —, realizada por outro mecanismo. Lá o endereço vinha do campo f da instrução; aqui vem de um fio. A ideia sobrevive à mudança de implementação, e é por isso que ela merecia um nome.

NOTA

1.1 A conta de pinos

Com a linha de leitura ligada permanentemente ao terra, o display consome:

Modo	Pinos	Composição
Oito bits	10	D0–D7, mais RS e E
Quatro bits	6	D4–D7, mais RS e E

Quatro pinos de diferença decidem o projeto do semestre. Com 34 pinos de E/S e a necessidade de acomodar sensor, aquecedor, ventoinha, teclado matricial e comunicação, o modo reduzido não é uma otimização: é o que torna o termostato viável no kit.

NOTA

Note também o que a conta **não** diz. É comum ouvir que o paralelo é mais rápido que o serial; não é, e o mundo dos periféricos modernos é a prova.

A distinção que importa é outra: **quem gera o protocolo**. Aqui é o software — cada borda de E é uma instrução sua, e o processador fica preso enquanto isso. No encontro 10 o mesmo trabalho passa a um periférico dedicado, que produz as bordas sozinho enquanto o programa faz outra coisa. Essa é a fronteira real, e ela não tem nada a ver com o número de fios.

O próprio PIC18F4550 tem um barramento paralelo **no outro papel**: a PORTD pode operar como porta paralela escrava, com linhas de seleção e de habilitação vindas de fora. Nessa configuração o microcontrolador é o periférico, e quem produz as bordas é outro processador.

NOTA

Mesmo contrato, papéis invertidos. Vale saber que existe, porque explica por que a PORTD tem funções alternativas com nomes estranhos na tabela de pinos.

2 O controlador HD44780

O display do laboratório não é uma matriz de pontos comandada diretamente: é um módulo com controlador próprio, que recebe comandos e caracteres e cuida da varredura, da geração dos desenhos e da atualização.

Elemento	Função
Memória de exibição (DDRAM)	Guarda os códigos dos caracteres mostrados. Cada posição da tela corresponde a um endereço fixo
Gerador de caracteres em ROM	Contém os desenhos dos caracteres padrão, indexados pelo código recebido
Gerador de caracteres em RAM	Permite definir caracteres próprios — tipicamente o símbolo de grau, ausente do conjunto padrão
Registrador de instrução (IR)	Recebe comandos: limpar, posicionar cursor, configurar modo
Registrador de dados (DR)	Recebe os códigos dos caracteres a exibir

E três linhas de controle governam a comunicação:

Linha	Nome usual	Significado
RS	Seleção de registrador	Nível baixo escolhe o IR (comando); nível alto escolhe o DR (dado)
R/W	Leitura ou escrita	Nível baixo escreve. Em geral ligado permanentemente ao terra, economizando um pino
E	Habilitação	O dado é capturado na borda de descida . É o pulso que efetiva a transferência

Aqui a montagem da placa decide o resto da aula: R/W está ligado ao terra. Verifique a serigrafia antes de assumir isso em outro hardware — em módulos ligados com R/W acessível, tudo o que vem a seguir muda.

NESTE KIT

Ligar R/W ao terra economiza um pino e tem uma consequência que atravessa o resto da aula: torna-se impossível **ler** o indicador de ocupado do controlador. Sem ele não há como perguntar se a operação anterior terminou, e resta esperar o tempo máximo especificado.

ATENÇÃO

Toda a temporização por atraso fixo descrita adiante é consequência dessa escolha de hardware — e não de uma limitação do HD44780, que sabe responder. No encontro 11 o mesmo problema reaparece com o I²C, e lá o dispositivo **avisa** quando terminou. Mesmo problema, duas soluções de qualidade diferente.

2.1 Endereçar a tela

Os endereços da memória de exibição não são contíguos entre as linhas.

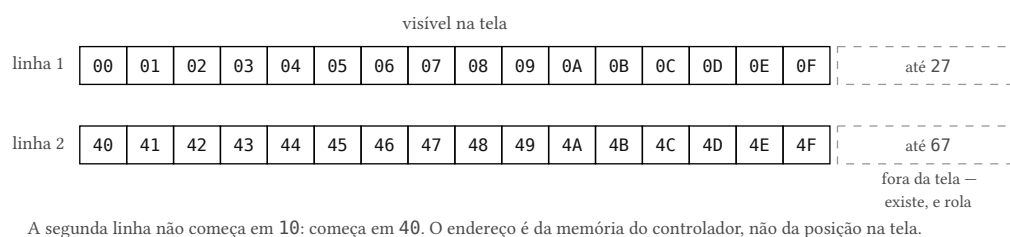


Figura 1: A memória do controlador é maior que a tela: são 40 posições por linha, das quais 16 aparecem. As demais existem, guardam o que for escrito nelas, e entram em cena com os comandos de rolagem.

A descontinuidade é a origem de um defeito clássico: escrever dezessete caracteres seguidos **não** faz o texto passar para a segunda linha — ele continua em posições invisíveis da primeira. Mudar de linha exige um comando explícito de posicionamento.

ATENÇÃO

3 O contrato, em números

A folha de dados do HD44780 especifica um ciclo de escrita com cinco tempos mínimos. Todos eles valem para o modo de quatro bits sem alteração: o que muda é que cada byte gasta dois ciclos em vez de um.

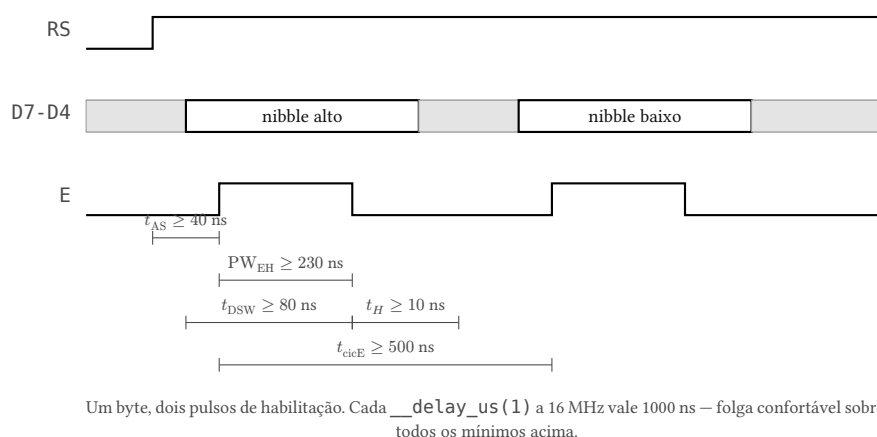


Figura 2: O ciclo de escrita de um byte em quatro bits. Os números são os mínimos da folha de dados a 5 V.

De onde sai o `__delay_us(1)`. A 16 MHz, um ciclo de máquina vale 250 ns, e `__delay_us(1)` consome quatro deles: 1000 ns. Compare com os mínimos:

CONCEITO

$PW_{EH} \geq 230 \text{ ns}$, $t_{AS} \geq 40 \text{ ns}$, $t_{DSW} \geq 80 \text{ ns}$, $t_H \geq 10 \text{ ns}$, $t_{cicE} \geq 500 \text{ ns}$.

O menor múltiplo de T_{cy} que satisfaz todos é **um único ciclo de máquina**, 250 ns — e ele já bastaria para quatro dos cinco. O `__delay_us(1)` é escolhido por outro motivo: dá folga de mais de quatro vezes sobre o pior caso, o que absorve variação de módulo, de temperatura e de tensão sem que ninguém precise medir nada.

Isto é diferente de copiar o valor de um exemplo da internet. O número foi escolhido **sabendo** qual restrição ele atende e com que margem.

Os 230 ns valem para operação a 5 V. Módulos alimentados a 2,7 V, e boa parte dos clones vendidos hoje, especificam $PW_{EH} \geq 450 \text{ ns}$. Ainda cabe no `__delay_us(1)`, mas não caberia num pulso de um ciclo de máquina.

DIVERGÊNCIA

Este é o argumento contra “otimizar” o driver reduzindo o pulso: o ganho é de 750 ns por nibble, e o preço é um driver que funciona na sua bancada e falha na do colega, com um módulo de outro lote.

4 O modo de quatro bits

Cada byte vai em duas etapas: primeiro os quatro bits mais significativos, depois os quatro menos significativos, cada metade com seu próprio pulso de habilitação.

```
static void lcd_pulso(void)
{
    LATCbits.LATC0 = 1;          /* E em nível alto      */
    __delay_us(1);               /* PW_EH: mínimo 230 ns */
    LATCbits.LATC0 = 0;          /* borda de descida: captura */
    __delay_us(1);               /* t_cicE: mínimo 500 ns entre subidas */
}

static void lcd_nibble(uint8_t valor)
{
    LATD = (uint8_t)((LATD & 0x0F) | (valor & 0xF0));
    lcd_pulso();
}

void lcd_escrever(uint8_t valor, uint8_t eh_dado)
{
    LATCbits.LATC1 = eh_dado;     /* RS */
    lcd_nibble(valor);            /* parte alta */
    lcd_nibble((uint8_t)(valor << 4)); /* parte baixa */
    __delay_us(40);              /* execucao: uma vez por byte */
}
```

A espera de execução vale por **byte**, não por nibble. O controlador só começa a executar quando o segundo nibble chega; esperar 40 µs depois do primeiro é esperar por nada.

ATENÇÃO

O erro é fácil de cometer — basta pôr o atraso dentro de `lcd_pulso`, onde ele parece pertencer — e dobra o custo de toda a interface. Uma tela inteira passa de 1,3 ms para 2,7 ms sem que nada pare de funcionar.

4.1 Por que LATD e não PORTD

A primeira linha de `lcd_nibble` lê LATD para preservar a parte baixa enquanto altera a parte alta. Trocar por PORTD reintroduz um defeito que o curso ainda não nomeou.

Leitura-modificação-escrita. Toda operação sobre um bit de porta — LATD |= 0x10, BSF, LATDbits.LATD4 = 1 — é na verdade três operações: ler os oito bits, alterar um, escrever os oito de volta.

CONCEITO

Se a leitura vem de PORTD, o que se lê é o **estado elétrico dos pinos**, não o que foi escrito neles. Um pino carregado por capacitância, por um LED, ou pela entrada de outro chip pode não ter chegado ao nível escrito no instante da leitura — e o valor lido, errado, é reescrito nos outros sete pinos.

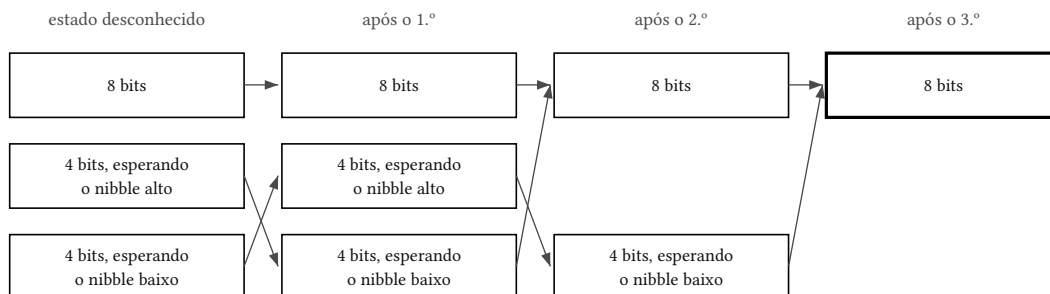
LATD lê o **latch**: o que o programa mandou, independentemente do que o mundo fez com o fio. É por isso que a família PIC18 tem LAT, e a família de faixa média não tinha.

Este defeito é primo do “LAT antes de TRIS” do encontro 2, e tem a mesma assinatura: nada trava, o pino errado muda, e o sintoma depende da carga elétrica ligada — ou seja, some quando você desconecta as coisas para investigar.

NOTA

4.2 A sequência de inicialização

O controlador nasce em modo de oito bits, mas **reiniciar o microcontrolador não reinicia o display**. Depois de um reset por SW9, com o módulo ainda alimentado, o controlador pode estar em qualquer um de três estados.



Cada seta é um pulso de 0x3 em D7–D4. O estado que espera o nibble baixo completa o byte 0x33 — que é justamente “interface de 8 bits”. Três repetições porque três é o número de estados possíveis, e o programa não sabe em qual deles começou.

Figura 3: A inicialização é uma resssincronização às cegas. O programa não pode perguntar em que estado o controlador está — só pode enviar uma sequência que leva os três ao mesmo lugar.

São **três** estados possíveis, não dois: oito bits; quatro bits esperando o nibble alto; quatro bits esperando o nibble baixo. O último existe porque um reset pode ocorrer entre os dois pulsos de um mesmo byte.

CONCEITO

Um pulso de 0x3 em D7–D4 age diferente em cada um:

em oito bits, é o comando “interface de 8 bits” — nada muda;

esperando o alto, vira o nibble alto — passa a esperar o baixo;

esperando o baixo, completa um byte com o nibble alto anterior, executa lixo, e volta a esperar o alto.

Só na **segunda** rodada o estado “esperando o baixo” completa o byte 0x33, que é exatamente “interface de 8 bits”. Daí três repetições: três estados, e a convergência custa três passos.

```
void lcd_iniciar(void)
{
    __delay_ms(50);           /* inicializacao interna do modulo */

    lcd_nibble(0x30); __delay_ms(5);
    lcd_nibble(0x30); __delay_us(150);
    lcd_nibble(0x30); __delay_us(150);
    lcd_nibble(0x20);         /* a partir daqui, modo de 4 bits */

    lcd_comando(0x28);        /* 4 bits, 2 linhas, matriz 5x8 */
    lcd_comando(0x0C);        /* display ligado, sem cursor */
    lcd_comando(0x06);        /* avanco automatico do cursor */
    lcd_comando(0x01);        /* limpar */
    __delay_ms(2);            /* limpar demora mais que os demais */
}
```

Os três atrasos da sequência têm origens diferentes, e nenhum é arbitrário.

CONCEITO

50 ms — não são para o display: são para a **alimentação**. O controlador só garante o estado interno depois que VDD estabiliza, e quem liga o kit pela USB não controla quanto tempo isso leva.

5 ms depois do primeiro 0x30 — porque o byte de lixo completado pelo estado “esperando o baixo” pode ser 0x03, que é o comando “retornar ao início” e custa cerca de 1,5 ms. Cinco milissegundos cobrem o pior caso com folga tripla. Este é o único atraso da sequência que existe por causa de um comando que **ninguém quis executar**.

150 µs nas duas repetições seguintes — a partir daí não há mais byte de lixo possível, e basta o tempo de um comando comum, com margem.

5 O custo em tempo

A maioria dos comandos exige cerca de 37 µs. Escrever dezesseis caracteres numa linha custa

CONCEITO

$$16 \cdot 37 \mu\text{s} \approx 0,6 \text{ ms}$$

Somando o posicionamento de cursor de cada linha e considerando as duas linhas, uma atualização completa de tela custa da ordem de **1,3 ms**.

O comando de limpeza, sozinho, exige cerca de 1,5 ms — quarenta vezes mais que um comando comum, porque ele percorre as 80 posições da memória de exibição.

Em ciclos de máquina, a 250 ns por ciclo:

Operação	Tempo	Ciclos	Comparação
Um comando ou caractere	40 µs	160	—
Atualização de tela	1,3 ms	5200	O mais caro do sistema
Limpar a tela	1,5 ms	6000	E ainda pisca
Uma conversão do ADC	15 µs	60	Afirmado aqui; derivado no encontro 4

O display é, com folga, o periférico mais lento do sistema — quase duas ordens de grandeza acima de tudo o mais. Qualquer discussão sobre desempenho neste projeto passa por ele, e por nenhum outro.

ATENÇÃO

A consequência é contraintuitiva o suficiente para valer como regra: **quem quiser mais tempo de processador não deve medir menos — deve escrever menos**.

5.1 Duas consequências de projeto

Não limpar a tela a cada atualização. Limpar e reescrever custa os 6000 ciclos do comando somados aos 5200 da escrita, e produz tremulação visível: por um instante a tela está em branco. A prática correta é reposicionar o cursor e sobrescrever, mantendo o comprimento do texto constante e completando com espaços quando o número encolhe — caso contrário restam dígitos antigos à direita.

Não atualizar mais rápido do que se consegue ler. Atualizar a cada 1 ms é desperdício puro: o olho não acompanha, e o custo é o mais alto do sistema. Uma atualização a cada 200 ms a 500 ms é confortável e reduz o consumo de tempo em duas ordens de grandeza.

```
/* Escrever apenas quando o valor mudou. */
static int16_t exibido = INT16_MIN;

void atualizar_display(int16_t decimos)
{
    if (decimos == exibido) {
        return; /* nada a fazer: sai imediatamente */
    }
    exibido = decimos;
}
```

```

    lcd_posicao(0, 0);
    lcd_texto("T=");
    lcd_numero_fixo(decimos);      /* largura constante */
    lcd_texto(" C ");
}

```

Guardar o que já foi mostrado e escrever apenas a diferença é a versão elementar de uma ideia que reaparece em toda interface gráfica, de displays de duas linhas a navegadores: **comparar com o estado exibido é sempre mais barato que redesenhar.**

NOTA

5.2 O que importa não é a média

Os 1,3% do Exercício 3.3 podem sugerir que o display é barato. A média engana.

O custo do display não é uma fração de tempo distribuída: é um **bloqueio contínuo** de 1,3 ms. Durante ele, o processador não faz nada além de esperar o controlador — não lê botão, não converte, não alterna pino.

CONCEITO

O número a comparar não é 1,3% do segundo. É 1,3 ms contra a menor coisa que o sistema precisa fazer a tempo.

O curso já tem esse número. O gerador de melodia da bancada produz um lá de 440 Hz invertendo um pino a cada meio período — **1,14 ms**.

Uma atualização de tela é mais longa que meio período do lá. Se as duas coisas convivem no mesmo laço, a nota sai errada toda vez que o display é escrito, e o defeito é audível: um estalo, ou a nota cai de tom.

ATENÇÃO

Nenhuma reorganização do laço resolve. Enquanto a escrita for bloqueante, existe um intervalo de 1,3 ms em que nada mais acontece — e a única saída é o processador ser **interrompido** durante a espera.

É a dívida que o encontro 7 paga.

6 Transposição

Displays modernos são majoritariamente seriais, e a economia de pinos é decisiva. Mudam a camada física e a velocidade; permanecem intactas a separação entre comando e dado e a posição do display como periférico lento do sistema.

Em plataformas com mais memória aparece uma diferença estrutural: o **quadro de memória**. Em vez de escrever caracteres direto no display, o programa desenha numa imagem em RAM e a transfere de uma vez, por acesso direto à memória, sem custo de processador.

NOTA

Isso exige memória que este dispositivo não tem — um display gráfico modesto consome mais RAM do que os 2 kB disponíveis — e é uma das fronteiras concretas entre 8 e 32 bits que os seminários dos encontros 14 e 15 vão revisar.

7 Previsão para o R4

PREVISÃO

P1. Você vai escrever `lcd_texto("Temp: 25.3 C")` e, sem limpar, escrever `lcd_texto("Temp: 9.1 C")`. Desenhe as dezesseis posições da linha 1 como você espera vê-las depois da segunda escrita.

P2. Se você trocar LATD por PORTD na primeira linha de `lcd_nibble`, o display continua funcionando na sua bancada? Diga sim ou não e justifique com o que está ligado ao PORTD do kit.

P3. Você vai remover uma das três repetições de 0x30 da inicialização. Em que condição o display falha, e em que condição ele funciona mesmo assim?

P4. Meça, com o osciloscópio no pino E, a largura do pulso de habilitação. Escreva o valor que espera antes de medir, e diga de onde ele vem.

P5. Estime o tempo total gasto com o display num sistema que atualiza as duas linhas a cada 100 ms, como percentual do tempo de processador. Repita para 500 ms.

Leve esta folha preenchida e a listagem do driver impressa ou aberta. A previsão é avaliada pelo raciocínio, não por acertar o número.

SEM NOTA

8 Exercícios

Exercício 3.1. Calcule quantos pinos o display consome em oito e em quatro bits, incluindo as linhas de controle e considerando R/W ligado ao terra. Em seguida verifique, no contrato de pinos do laboratório, se a versão de oito bits seria viável junto aos demais periféricos do projeto.

TAREFA

Exercício 3.2. Um estudante escreve `lcd_texto("Temp: 25.3 C")` e, em seguida, `lcd_texto("Temp: 9.1 C")`, sem limpar a tela. Descreva exatamente o que aparece no display e proponha duas correções: uma na função de escrita e outra na formatação do número.

TAREFA

Exercício 3.3. Estime o tempo gasto com o display num sistema que atualiza duas linhas completas a cada 100 ms, e expresse como percentual do tempo de processador. Repita para 500 ms e comente a diferença.

TAREFA

Exercício 3.4. Explique por que `lcd_nibble` lê LATD em vez de PORTD, e descreva concretamente o defeito que ocorreria com a leitura da porta caso os quatro pinos baixos estivessem ligados a uma carga capacitiva.

TAREFA

Exercício 3.5. A sequência de inicialização envia três vezes o mesmo valor antes de comutar o modo.

TAREFA

(a) Quantos estados distintos o controlador pode ocupar quando o microcontrolador é reiniciado sem que o display seja desenergizado? Liste-os.

(b) Mostre, para cada estado, o que acontece a cada um dos três pulsos.

(c) Duas repetições bastariam? E quatro seriam mais seguras?

Exercício 3.6. O driver mostrado no início desta aula tinha a espera de 50 μ s dentro de `lcd_pulso`, e não uma espera de 40 μ s por byte.

TAREFA

(a) Quanto custava um byte na versão antiga e quanto custa na corrigida?

(b) Qual era o tempo de uma atualização completa de tela na versão antiga?

(c) A derivação de 1,3 ms da seção 7 valia para qual das duas versões?

Exercício 3.7. Um estudante propõe: “se o display é o gargalo, é só aumentar o clock do processador”. Avalie a proposta. O que melhora, o que não melhora, e por quê?

TAREFA

No encontro 4: aquisição analógica. O número afirmado na tabela desta aula — 60 ciclos por conversão — sai de uma conta com o tempo de aquisição e o tempo de decisão do conversor, e o caminho até ele passa por uma descoberta desconfortável: o pino, que este curso vinha tratando como um bit desde o encontro 0, não é um bit.

NOTA